

초급 프로젝트

최종 발표

알약 객체 검출 모델 제작

codeit;

팀 파이리(1팀)

엄재웅(팀장)

신재민

김라희

윤성원

프로젝트 기간

2026.09.10 ~

2026.09.29

🔍 목차

1 설계

- 모델 선택
- 데이터
- 전처리

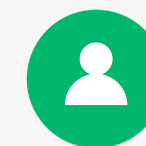
2 실험

- 기본설정 수정
- 데이터분할
- 미세조정
- Alhubdata 추가

3 정리

- 프로젝트 관리
- 회고

설계



발표자
- 엄재웅

설계

- 데이터 구성
- 필요 정보 추출
- 박스 범위 체크
- 라벨 검사
- 모델 선택
- 라벨 형식 비교
- 박스 좌표 변경
- 알약 ID 재매핑
- 모델 학습 및 추론

🔍 설계 - 데이터 구성

train_images



train_annotations

```
"images": [  
  {  
    "file_name": "K-001900-016548-019607-029451_0_2_0_2_70_000_200.png",  
    "width": 276,  
    "height": 1280,  
    "imgfile": "K-001900-016548-019607-029451_0_2_0_2_70_000_200.png",  
    "drug_N": "K-001900",  
    "drug_S": "정신질환",  
    "back_color": "연회색 배경",  
    "drug_dir": "앞면",  
    "light_color": "주백색",  
    "camera_la": 70,  
    "camera_lo": 0,  
    "size": 200,  
    "dl_idx": "1900",  
    "dl_mapping_code": "K-001900",  
    "dl_name": "보령부스파정 5mg",  
    "dl_name_en": "Buspar Tab. 5mg Boryung",  
    "img_key": "http://connectdl.com/design/img/drug/14xwks5v01l.jpg",  
    "dl_material": "부스피론염산염",  
    "dl_material_en": "Buspirone Hydrochloride",  
    "dl_custom_shape": "정제, 저장정",  
    "dl_company": "보령제약(주)",  
    "dl_company_en": "Boryung",  
    "dl_company_mf": "",  
    "dl_company_mf_en": "",  
    "item_seq": 198700706,  
    "di_item_permit_date": "19870323",  
    "di_class_no": "[01170]정신신경음제",  
    "di_etc_etc_code": "정문의약품",  
    "di_edl_code": "841901280, A09002281",  
    "chant": "이약은 양면블록한 강방형의 흰색정제이다",  
    "drug_shape": "경방형",  
    "thick": 2.5,  
    "leng_long": 0,  
  },  
]
```

test_images



🔍 설계 - 필요 정보 추출

```
for f in raw.glob("train_annotations/**/*.json"):
    data = json.loads(f.read_text(encoding="utf-8"))

    img = data["images"][0]
    x, y, w, h = data["annotations"][0][
        "bbox"
    ]
    cat = data["categories"][0]

    by_image[img["file_name"]].append(
        {
            "x": x,
            "y": y,
            "w": w,
            "h": h,
            "class_id": cat["id"],
            "class_name": cat["name"],
        }
    )

return dict(by_image)
```

JSON 파일 순회 및 파싱
이미지 정보, 바운딩 박스, 클래스 추출

딕셔너리 구조화
이미지 파일명 기준으로 객체 정보 통합

🔍 설계 - 바운딩 박스 범위 체크

```
def is_valid(b):  
    return (  
        b["x"] >= 0  
        and b["y"] >= 0  
        and b["w"] > 0  
        and b["h"] > 0  
        and b["x"] + b["w"] <= IMG_W  
        and b["y"] + b["h"] <= IMG_H  
    )
```

바운딩 박스 유효성 검증

좌표가 이미지 범위 안에 있는지, 폭/높이가 0보다 큰지 검사

※ 기존 232장 중 1장 제외

🔍 설계 - 바운딩 박스 범위 체크

```
"annotations": [  
  {  
    "area": 97965,  
    "iscrowd": 0,  
    "bbox": [  
      6567,  
      625,  
      311,  
      315  
    ],  
    "category_id": 18357,  
    "ignore": 0,  
    "segmentation": [],  
    "id": 903,  
    "image_id": 239  
  },  
],
```



✓ bbox x좌표(6567)가 이미지 크기 범위를 벗어난 오류 데이터

🔍 설계 - 라벨 검사(누락)

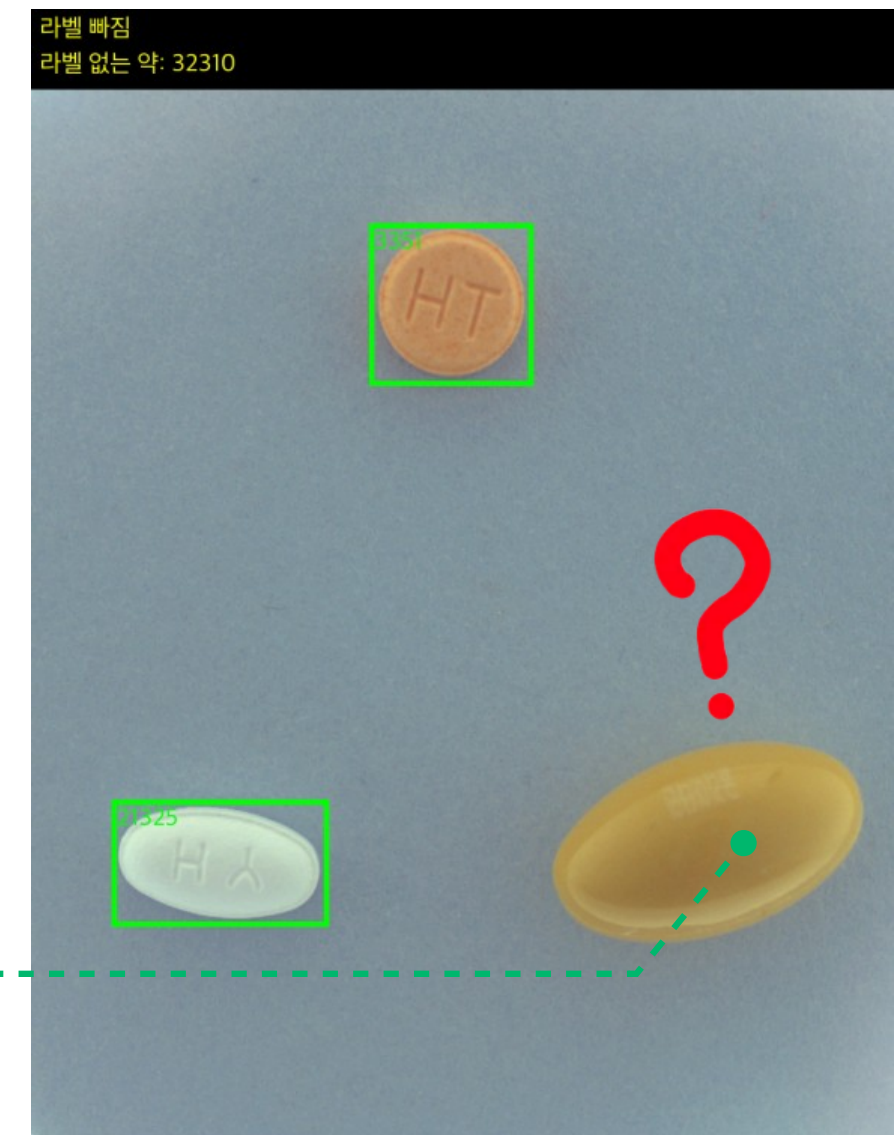
```
def has_missing_label(name, boxes):  
    labeled = []  
    for b in boxes:  
        labeled.append(b["class_id"])  
  
    for code in get_codes(name):  
        if code not in labeled:  
            return True  
  
    return False
```

이미지에 필요한 클래스가 모두 라벨링됐는지 검사
누락된 클래스가 하나라도 있으면 해당 이미지 제외 대상

※ 기존 231장 중 8장 제거

🔍 설계 - 라벨 검사(누락)

```
✓ K-003351
  {} K-003351-021325-032310_0_2_0_2_75_000_200.json
  {} K-003351-021325-032310_0_2_0_2_90_000_200.json
✓ K-021325
  {} K-003351-021325-032310_0_2_0_2_75_000_200.json
  {} K-003351-021325-032310_0_2_0_2_90_000_200.json
✓ K-032310
  {} K-003351-021325-032310_0_2_0_2_75_000_200.json
> K-003351-021325-033880_json
```



✓ K-032310 라벨 중 카메라 위도 90도에 해당하는 json이 누락된 오류 데이터

🔍 설계 - 라벨 검사(중복)

```
def compute_iou(a, b):  
    left = max(a["x"], b["x"])  
    top = max(a["y"], b["y"])  
    right = min(a["x"] + a["w"], b["x"] + b["w"])  
    bottom = min(a["y"] + a["h"], b["y"] + b["h"])  
  
    inter = max(0, right - left) * max(0, bottom - top)  
    union = a["w"] * a["h"] + b["w"] * b["h"] - inter  
  
    if union <= 0:  
        return 0.0  
    return inter / union
```

두 박스의 교집합/합집합 비율(IoU)로 겹침 정도 계산
IoU가 임계값(0.9) 이상이면 동일 객체의 중복 라벨링으로 판단

※ 기존 223장 중 3장 제거

🔍 설계 - 라벨 검사(중복)

```
"annotations": [  
  {  
    "area": 76494,  
    "iscrowd": 0,  
    "bbox": [  
      88,  
      255,  
      366,  
      209  
    ],  
    "category_id": 16548,  
    "ignore": 0,  
    "segmentation": [],  
    "id": 4683,  
    "image_id": 1228  
  }  
],  
"categories": [  
  {  
    "supercategory": "pill",  
    "id": 16548,  
    "name": "가바토파정 100mg"  
  }  
]
```

```
"annotations": [  
  {  
    "area": 76494,  
    "iscrowd": 0,  
    "bbox": [  
      88,  
      255,  
      366,  
      209  
    ],  
    "category_id": 33009,  
    "ignore": 0,  
    "segmentation": [],  
    "id": 4691,  
    "image_id": 1228  
  }  
],  
"categories": [  
  {  
    "supercategory": "pill",  
    "id": 33009,  
    "name": "신바로정"  
  }  
]
```



✓ 동일 bbox 좌표에 다른 카테고리(신바로정 / 가바토파정 100mg) 라벨이 중복 부여된 오류

🔍 설계 - 모델 선택

Faster R-CNN

대략적 후보를 뽑고(1차) → 후보만 정밀하게 재검토(2차)

- 1 백본 통과
입력을 이미지 분류 모델에 넣어서 특징 맵 추출
- 2 RPN으로 후보 영역 제안
특징 맵에서 '객체가 있을 것으로 예상되는
후보 박스 생성 (배경 vs 객체 여부만 판단)
- 3 후보 영역 잘라내기
제안된 후보 영역만 잘라서 크기 통일
- 4 각 후보를 정밀 판별
고정 크기로 맞춘 영역을 신경망에 통과시켜,
'클래스'와 '박스 좌표'를 판별

mAP75-95: 0.8599 / 학습 시간: 61분8초

RetinaNet

모든 위치를 검사하여 발생하는 문제를
Focal Loss로 해결

- 1 백본 통과
이미지 분류 모델에 넣어 특징 추출
- 2 FPN으로 다중 해상도 특징 맵 생성
여러 크기의 특징 맵을 만들어 다양한 객체를 탐지
- 3 모든 위치에서 한번에 판별
후보를 별도로 찾지 않고,
모든 위치에서 클래스와 bbox를 동시에 판별
- 4 Focal Loss로 학습
쉬운 배경의 영향력 ↓
식별이 어려운 객체 학습에 집중

mAP75-95: 0.9584 / 학습 시간: 33분28초

YOLO

이미지 전체를 한 번만 보고 격자 단위로 즉시 예측

- 1 백본 통과
이미지를 CNN에 통과시켜 특징 추출
- 2 이미지를 그리드로 분할
이미지를 격자로 나누고,
각 영역에서 객체 중심 위치를 판단
- 3 각 칸에서 한번에 예측
각 격자에서 bbox + 클래스 + 신뢰도를 동시에 예측
- 4 중복제거(NMS)
중복으로 탐지된 박스 중
신뢰도가 가장 높은 것만 남기고 제거

mAP75-95: 0.9451 / 학습 시간: 14분37초

🔍 설계 - 라벨 형식 비교

COCO

데이터셋 전체가 하나의 큰 JSON파일

박스의 좌 상단 모서리 (Top-Left)

이미지 해상도 기준의 실제 pixel값

고유 약품코드(1900, 2483 ... 비연속)

파일 형태

좌표 기준

좌표 단위

클래스 번호

YOLO

이미지 1장 당 1개의 .txt파일이 1:1매칭 개별 생성

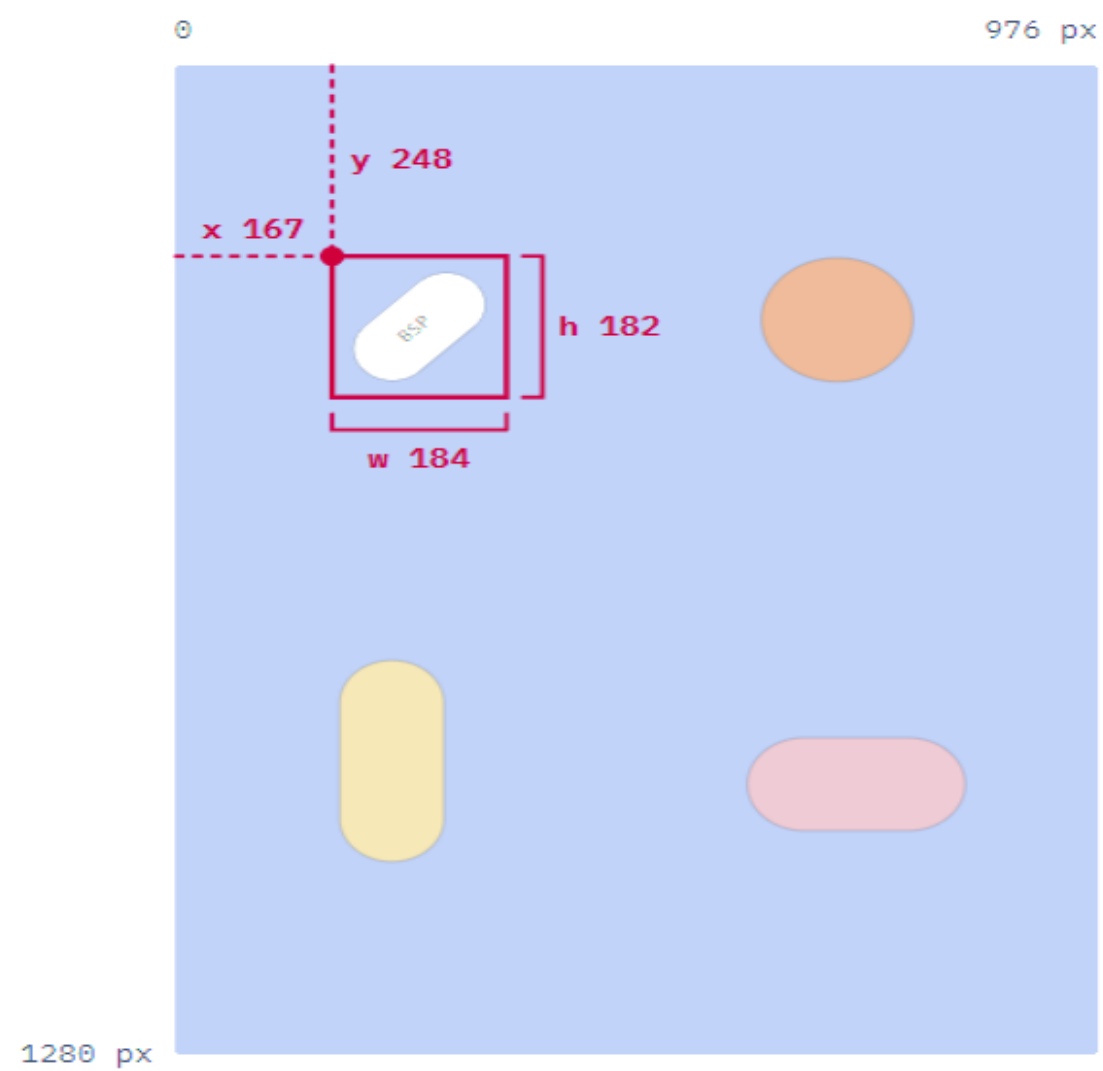
박스의 정중앙 (Center)

0 ~ 1 사이 상대적 비율 (Normalized)

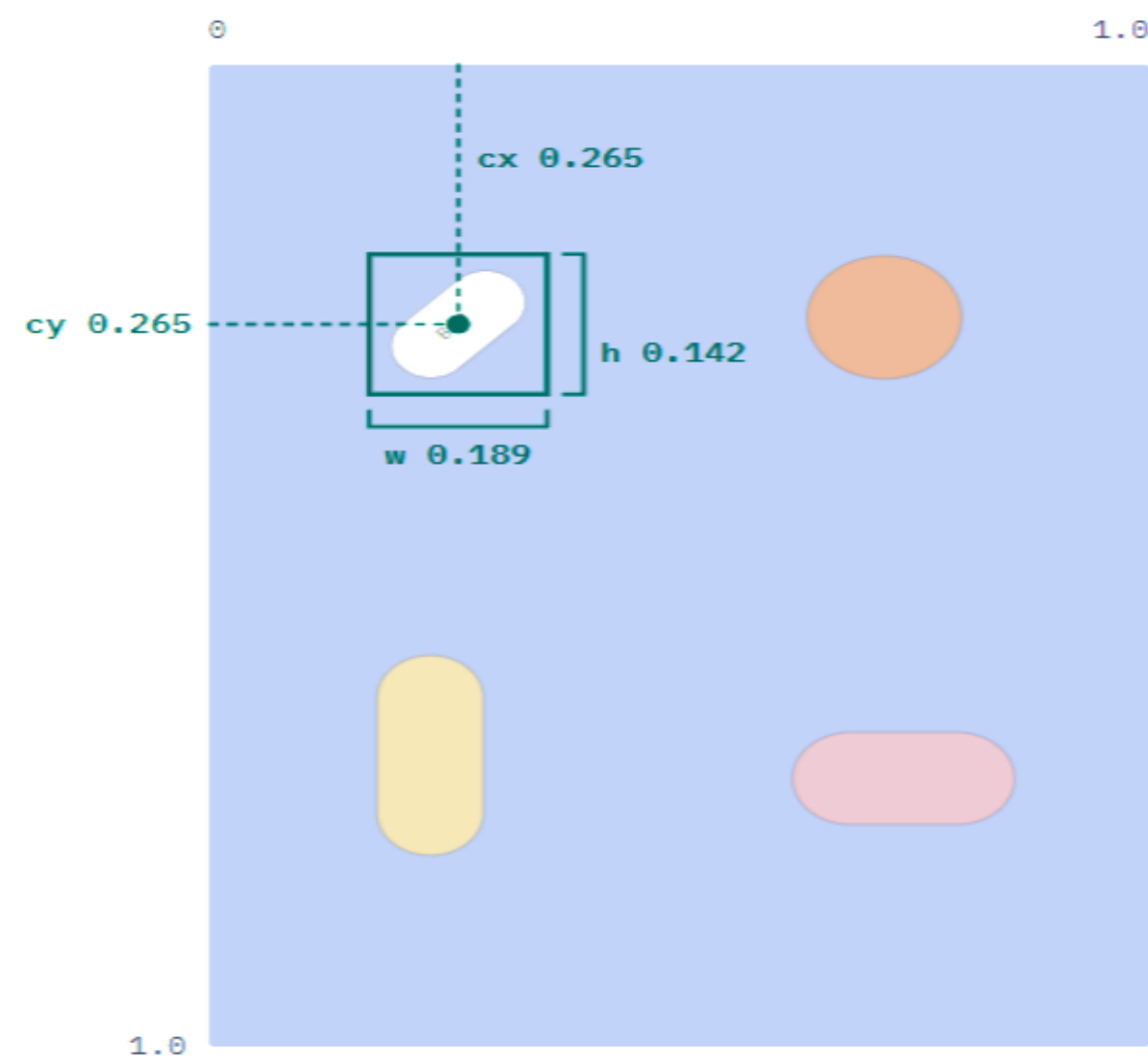
0번부터 이어지는 연속번호

🔍 설계 - 라벨 형식 비교

COCO



YOLO



🔍 설계 - 박스 좌표 변경

```
def to_yolo_line(b, idx):  
    cx = (b["x"] + b["w"] / 2) / IMG_W  
    cy = (b["y"] + b["h"] / 2) / IMG_H  
    w = b["w"] / IMG_W  
    h = b["h"] / IMG_H  
  
    return f"{idx} {cx:.6f} {cy:.6f} {w:.6f} {h:.6f}"
```

✓ 좌상단 기준 (x, y)를 중심점 (cx, cy)로 전환 이미지 해상도로 나누어 0~1 사이값으로 정규화

🔍 설계 - 알약 ID 재매핑

```
def save_yaml(class_ids):  
    lines = [  
        f"path: {YOLO_DIR}",  
        "train: images/train",  
        "val: images/val",  
        "names:",  
    ]  
  
    for i, class_id in enumerate(class_ids):  
        lines.append(f"  {i}: '{class_id}'")  
  
    (YOLO_DIR / "data.yaml").write_text("\n".join(lines) + "\n", encoding="utf-8")
```



비연속적인 알약 ID를 0부터 연속된 클래스 인덱스로 재매핑하여 YOLO 학습에 사용할 수 있도록 저장

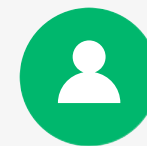
🔍 설계 - 모델 학습 및 추론

```
model = YOLO("yolo26n.pt")
model.train(
    data="data.yaml",
    epochs=50,
    imgsz=640,
    batch=16
)

metrics = model.val()
results = model.predict(source="test_image.jpg", conf=0.01)
```

✓ 기본 하이퍼파라미터로 YOLO 모델 학습 진행, 학습된 모델로 성능 평가(val) 및 테스트 이미지 추론(predict) 수행

실험

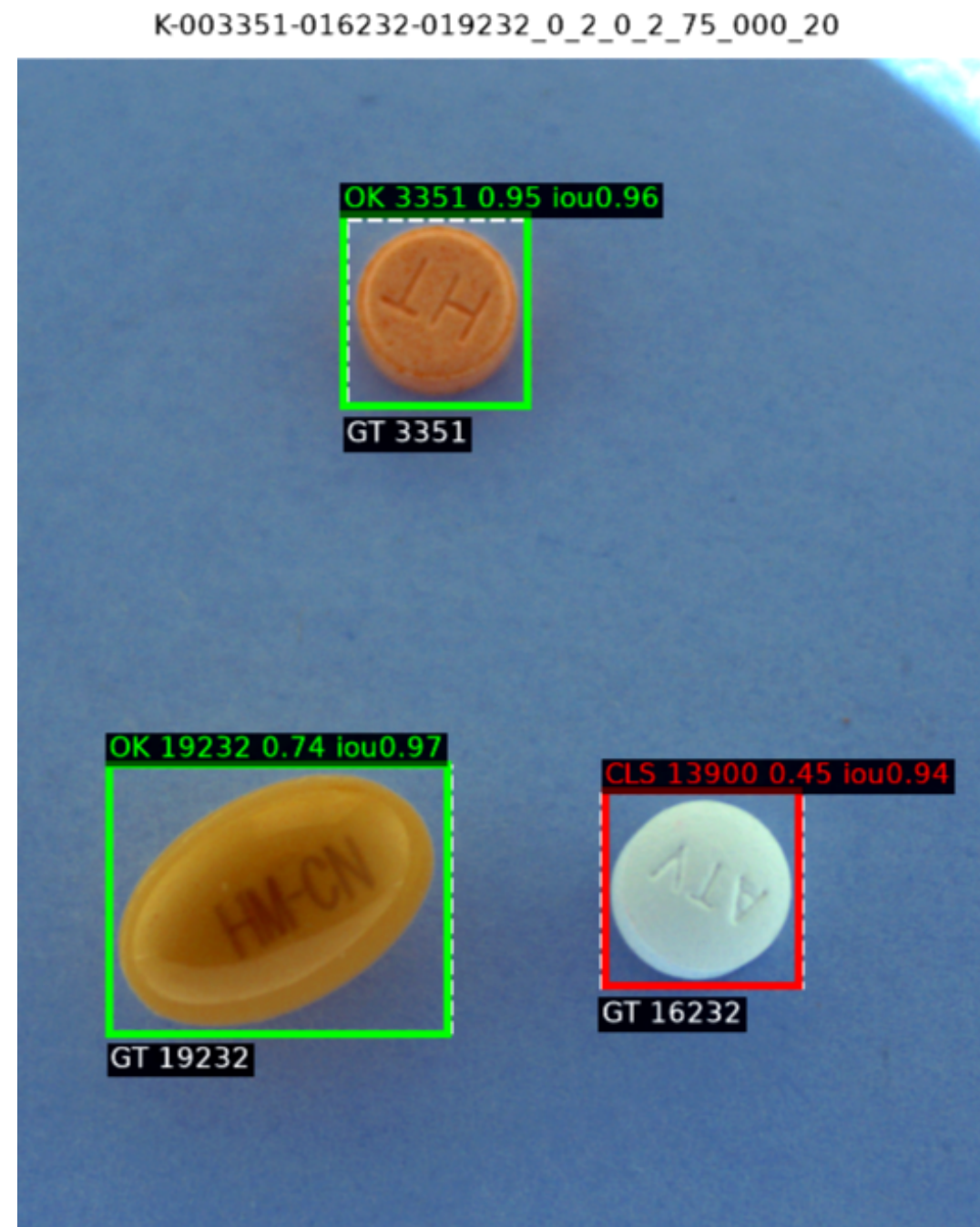


발표자
- 김라희, 신재민

실험

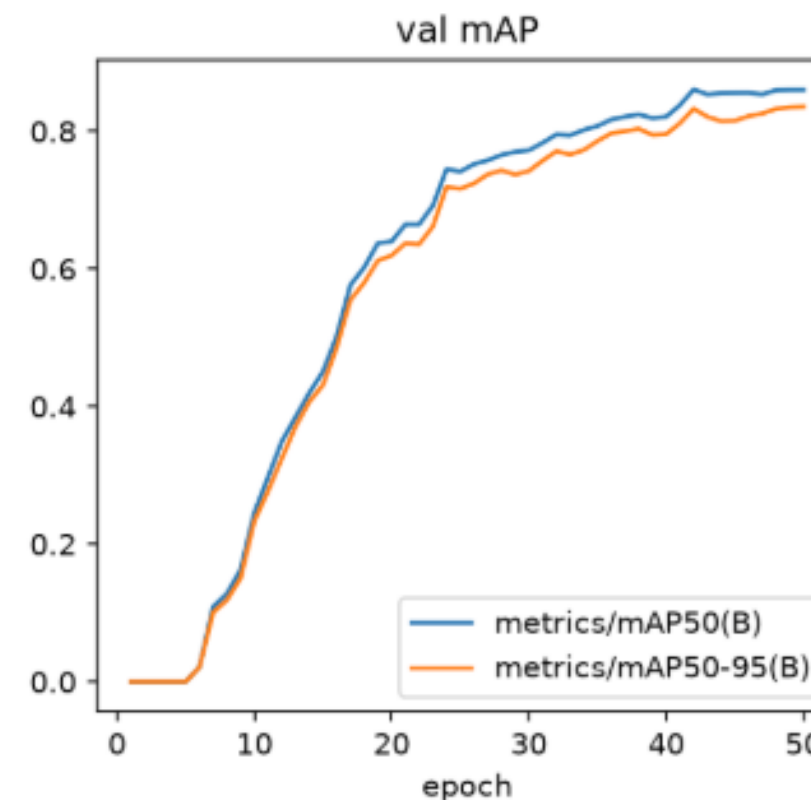
- 기본 설정 실험
- Epoch100 & lr 0.001
- train, val 재분할
- 이미지 사이즈 1280
- cos_lr True
- 하이퍼파라미터의 한계
- AI Hub 데이터 추가
- 오버샘플링 +
WBF 앙상블

🔍 기본 설정 - 실패 사례



- ❌ GT16232 - 실패 사례
위치 탐지는 정확함 (IOU 0.94)
정답 대신 유사 외형의 13900으로 오분류
신뢰도(CONF) 0.45로 낮음

🔍 Epoch100 & lr 0.001 - 가설 및 결과



1 해상도

동일한 형태, 색상의 알약은 각인으로 구분
해상도 640 → 960

2 학습 부족

50 epoch에서도 val이 계속 상승 판단
epoch 50 → 100

mAP 0.7179 → **0.9321** | Kaggle 0.235552 → **0.39139**

🔍 Epoch100 & lr 0.001 - 실패 사례



GT15607 - 실패 사례

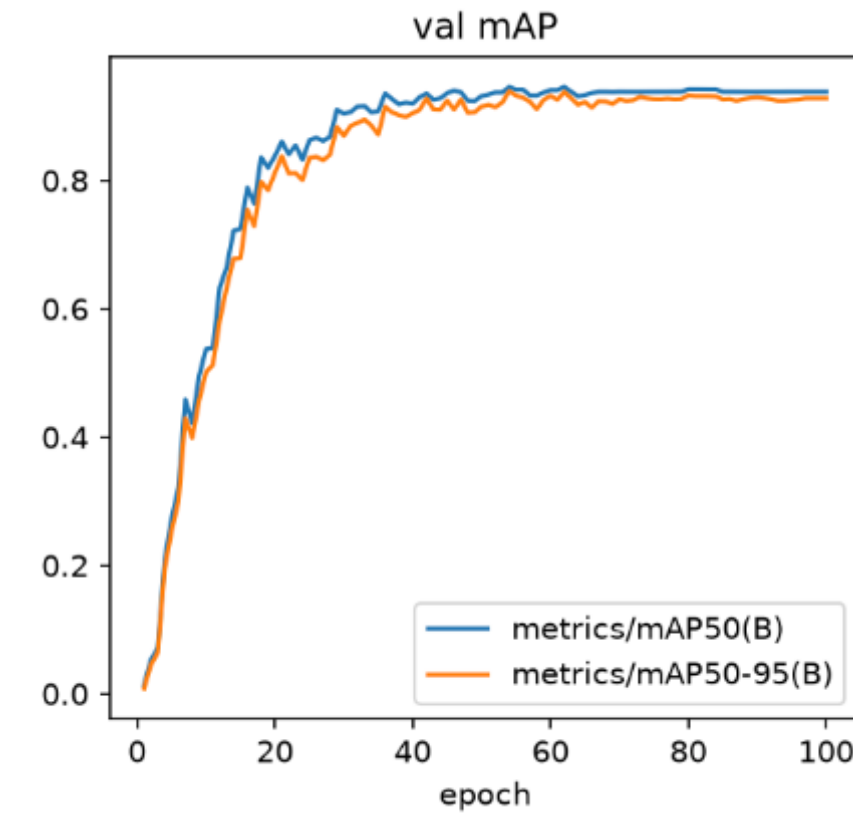
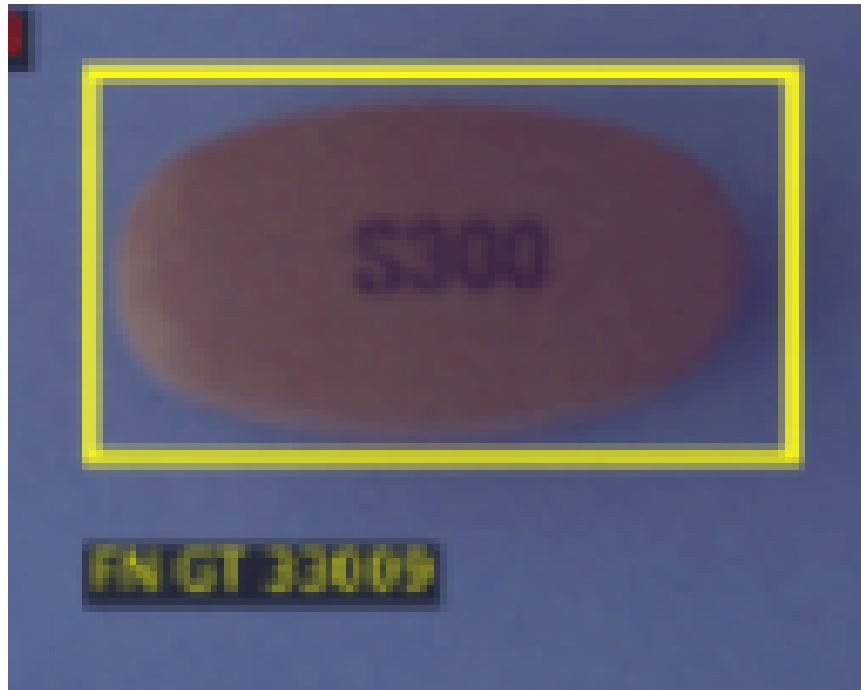
- 위치 탐지는 정확함 (IOU 0.98)
- 신뢰도(CONF) 0.42로 낮음
- 14232으로 오분류



GT 33009 - 실패 사례

- 예측 박스 없음
- 해당 알약을 탐지 못함

🔍 train, val 재분할 - 가설 및 결과



❶ 학습에 없는 클래스

33009는 train 데이터에 해당 클래스가 없음

-> val 재분할을 통해 모든 클래스가 train에 포함되도록 조정

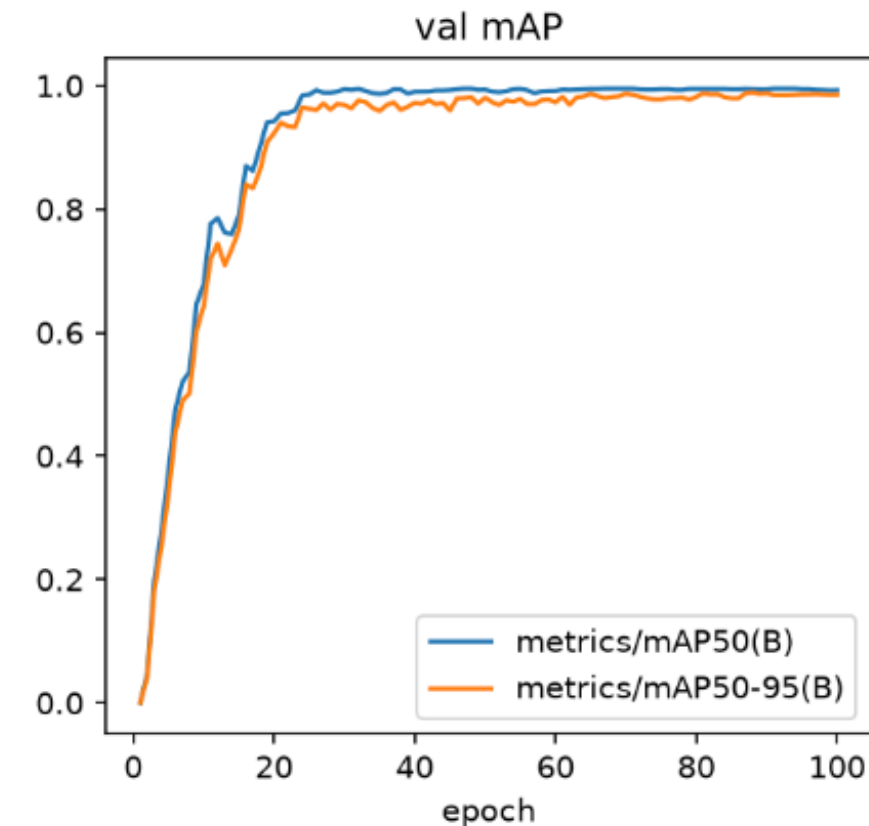
mAP 0.9321 -> 0.984 | Kaggle 0.39139 -> 0.41503

🔍 train, val 재분할 - 실패 사례



GT2966 TRAIN, VAL 재분할7 - 실패 사례
위치 탐지는 정확함 (IOU 0.95)
유사 외형의 16548(가바토파정)로 오분류
신뢰도(CONF) 0.55로 낮음

🔍 이미지 사이즈 1280 - 가설 및 결과



① 유사 외형 오분류

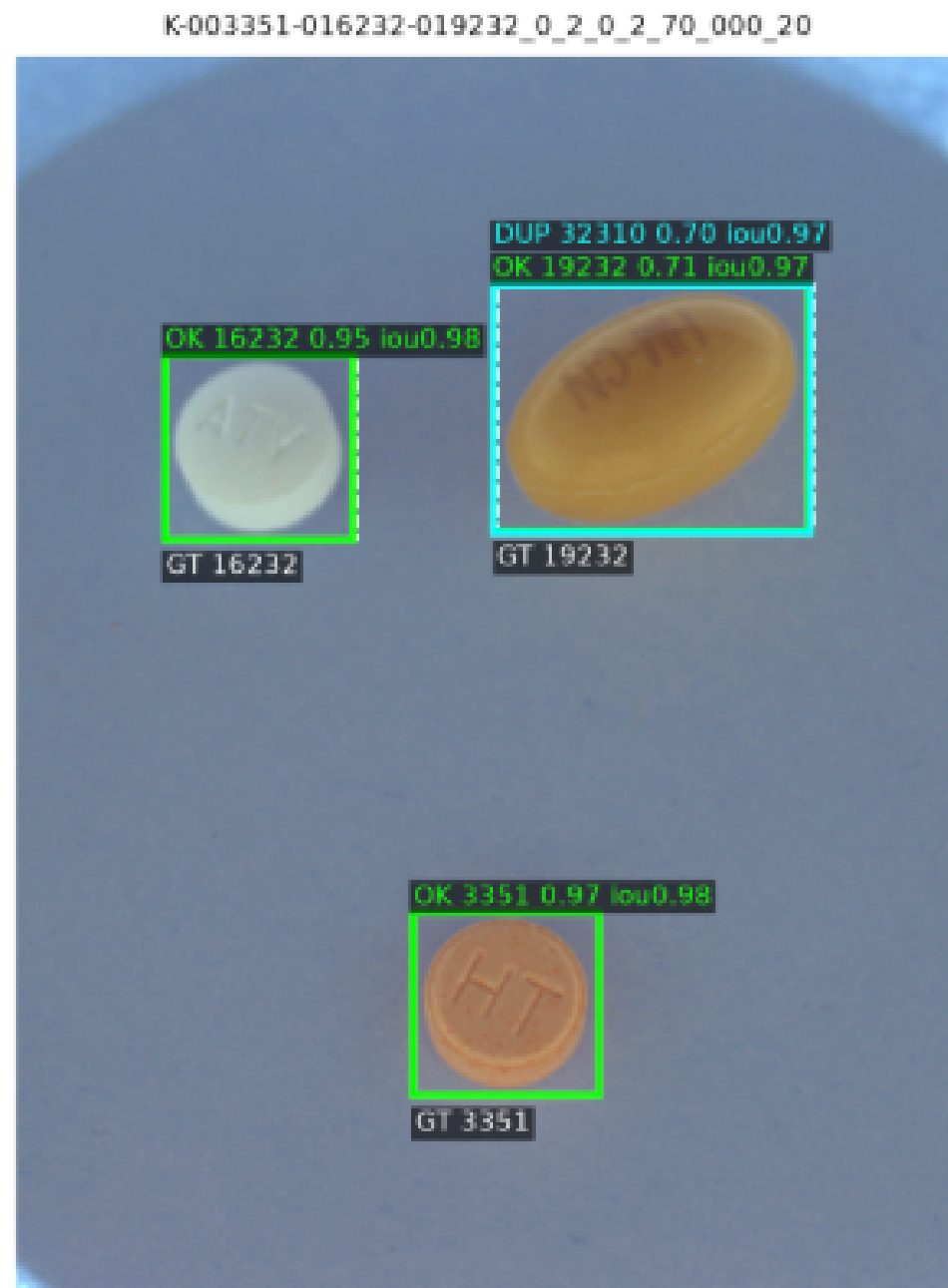
리바로정과 가바토파정은 형태, 색상, 크기가 유사하여 각인으로만 구분

960px에서는 각인 정보가 일부 픽셀로만 표현됨

→ 해상도를 높이면 각인 식별력 향상될 것으로 추측 **imgsz 960 -> 1280**

mAP 0.984 -> **0.984** | Kaggle 0.41503 -> **0.40805**

🔍 이미지 사이즈 1280 - 실패 사례



❌ GT19232 - 실패 사례 (중복 예측)
정답 예측은 맞음 (CONF 0.71, IOU 0.97)
글리아타민 예측이 하나 더 붙음



콜리네이트

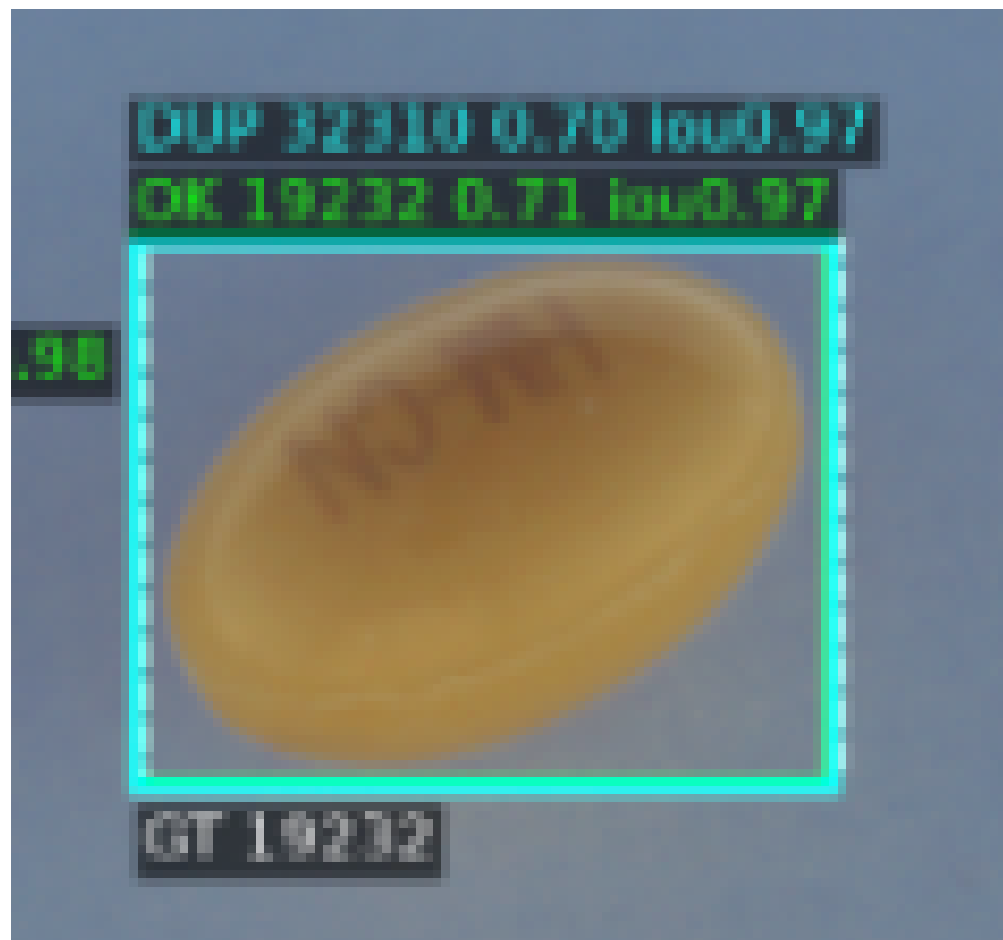


글리아타민



종근당 글리아타민

🔍 cos_lr True - 가설 및 결과



1 유사 외형 확신 부족

오분류·미탐지는 사라졌지만, 한 알약에 두 클래스가 동시에 살아남음
기본 lr(선형 감쇠)은 후반에도 보폭이 커서 결정 경계가 덜 다듬어진다고 추측
→ 후반 lr을 코사인으로 더 빠르게 줄여 알약 구분 정확도를 높일 것이라 예상
cos_lr False → True

mAP 0.984 → **0.9784** | Kaggle 0.40805 → **0.4155**

🔍 하이퍼파라미터의 한계

Kaggle	실험	바꾼 측	val mAP
0.41738	imgsz 1280	해상도	0.995
0.41550	cos_lr	lr 스케줄	0.9784
0.41504	val 재분할	데이터 분할	0.984
0.41467	yolo26s	모델 크기	0.981
0.40516	freeze 10	전이 학습	0.9738
0.40316	cls-pw 0.5	손실 가중치	0.9776
0.39132	mosaic 0.5	증강	0.976

✓ 7개 측을 전부 바꿔도 폭은 0.026 — 그럼 문제는 데이터인가?

🔍 하이퍼파라미터의 한계

1 0.41의 벽

- `imgsz, cos_lr, cls_pw, freeze, mosaic`, 모델크기(26n→26s) 전면 변경
- kaggle 점수 0.39132 ~ 0.41738(폭 0.026) 순위만 바뀔 뿐 한계가 존재
- val mAP는 전부 0.97 이상

2 원인

- Test에 Train의 56종에 포함 되지 않은 알약이 존재
모르는 알약은 어떤 하이퍼파라미터로도 맞출 수 없음
- 수렴(Optimization)이 아닌 정보 부족

3 방향성 전환

- 손덜 곳은 학습 설정이 아닌 데이터

🔍 AI hub data 추가

1 가설

test에 train 56종에 없는 알약이 존재 할 경우, 모델에 그 알약들을 학습 하이퍼파라미터로는 못 만드는 "정보"를 데이터로 직접 추가

2 실행 — AI Hub 「경구약제 이미지 데이터」(datasetkey 576) 조합 데이터 추가

- train 183장 56종 → 10,394장 118종 (AI Hub 10,211장 추가), val 37장은 유지
- TL_2_조합 · TS_2_조합은 대회 train/test 원본이라 제외
- 데이터가 56배 늘어 epochs 100 → 30

mAP 0.9784→ **0.9929** | Kaggle 0.41550 → **0.61074**

🔍 오버샘플링 + WBF 앙상블

1 오버샘플링 - 못 외운 알약 반복 학습

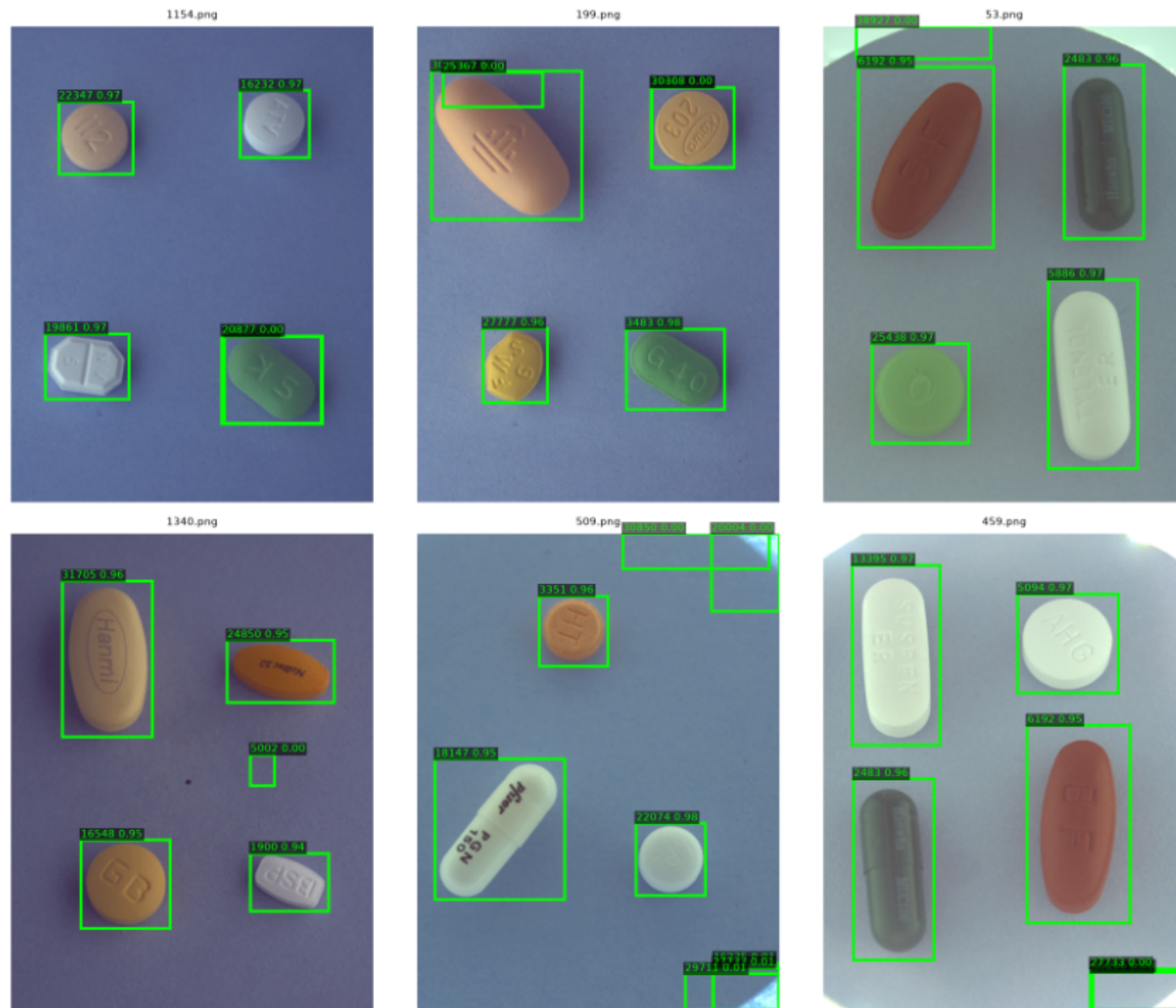
- 118종 중 3832만 혼자 뒤쳐짐 (AP 0.932)
해당 알약 데이터를 2배로 복제해 학습 빈도 증가
(새로운 사진을 만든 것이 아니라 같은 이미지를 더 자주 보여준 것)
- 0.61084 → 0.61802, 3832는 AP 0.995로 완전 회복

2 WBF 앙상블 - 두 모델 답을 합치기

- 모델 A(1280) + 모델 B(960) — 조건이 달라 서로 다른 실수를 발생
기존 방식(NMS)은 겹친 박스 중 하나만 선택하고 나머지는 제거
- WBF는 여러 박스의 위치를 평균 내서 새 박스 생성 → 박스 위치를 더 정확하게 조정
- 더 잘하는 B에 비중을 더 줌: 반반(1:1) 0.62054 → 1:1.3에서 0.62550

mAP 0.9929 → 0.9892 | Kaggle 0.61074 → 0.6255

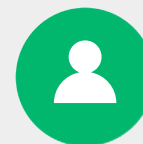
🔍 오버샘플링 + WBF 앙상블 - 테스트 결과



(842장 중 랜덤 6장, 신뢰도 0.25 이상만 표시)

- 알약이 누락 없이 검출
- 비슷한 알약들도 서로 다른 이름으로 구분
- 이미지 가장자리 여백에 불필요한 박스가 생성되나, 낮은 신뢰도로 점수 영향은 미미함
- 서비스 사용시 노이즈 필터링 로직 보완이 필요
- test는 정답이 공개되지 않아 정답 유무 모름
→ 눈으로만 확인

정리



발표자
- 윤성원

정리

- 프로젝트 관리
- 회고



3 문서 자동 갱신

시트 기록을 GitHub Actions가 읽음
-> README 그래프를 다시 그림
-> PR로 올려 팀장이 확인 후 Merge

Rank	Model Name	Score
5위	jw_ep30_img960_batch8_aihudata_oversample_f2 - 엄재웅	0.6107
4위	rahui_ep50_img128_batch8_aihudata_oversample_f2 - 김라희	0.6108
3위	rahui_ep50_img128_batch8_aihudata_oversample_f2 - 김라희	0.6171
2위	jw_ep30_img960_batch8_aihudata_oversample_f2 - 신재민	0.6207
1위	rahui_ep30_img960_batch8_aihudata_oversample_f2 - 김라희	0.6255

🔍 회고



**데이터는
눈으로 봐야 한다**



**모델도 여러 버전을
써봤어야 한다**



**로컬 점수를
믿으면 안 된다**



**결과물 관리가
중요하다**

Thank You

감사합니다

codeit;

팀 파이리(1팀)

엄재웅(팀장)

신재민

김라희

윤성원

프로젝트 기간

2026.09.10 ~

2026.09.29

